MARLUP: Actividad 3 – Desarrollo de sistema Processor-in-the-Loop

Aarón Salas Alvarado, Mónica Solís Murillo, Eduardo Valle Arriola
Instituto Tecnológico de Costa Rica

***Resumen*—MARLUP es una plataforma de nivelación accionada por cilindros hidráulicos. Este trabajo valida su controlador de posición mediante un esquema *Processor-in-the-Loop* (PiL). El modelo dinámico de un actuador, compuesto por un cilindro Bosch Rexroth CDH1 40/28 y una válvula proporcional 4/3, se simula en Simulink. El controlador PI, sintonizado manualmente ($K_p = 60$, $K_i = 1$, saturación en ± 1) y discretizado por Tustin con $T_s = 10\,\mathrm{ms}$, se ejecuta en un Arduino UNO R3. Ambos intercambian la posición y el comando por UART mediante una trama binaria de seis bytes. Se realizaron dos implementaciones. En la primera, sensores y actuadores son ideales; en la segunda, se incorporan no idealidades tomadas de las hojas de datos: cuantización del ADC, error de linealidad e histéresis del sensor magnetostrictivo. Frente a la simulación MiL, el PiL ideal presenta un error RMS de $0{,}0036\,\mathrm{mm}$, mientras que el caso no ideal alcanza un sobreimpulso de $3{,}47\,\%$ y un error en estado estacionario de $0{,}058\,\mathrm{mm}$, frente al requisito de $2{,}3\,\mathrm{mm}$. La degradación se atribuye principalmente a la cuantización del ADC, que por sí sola produce un error RMS de $0{,}113\,\mathrm{mm}$ respecto al MiL.**

***Index Terms*—Processor-in-the-Loop, control hidráulico, controlador discreto, no idealidades, MARLUP.**

I. Introducción

MARLUP es una plataforma de nivelación que controla el *roll* y el *pitch* de su mesa mediante cilindros hidráulicos, de forma similar a las plataformas paralelas de compensación de movimiento [6]. En la etapa anterior se diseñó, sobre un modelo dinámico del actuador, un controlador PI de posición que cumple los requisitos de sobreimpulso menor al $10\,\%$ y error de nivelación menor a $0{,}2°$, equivalente a unos $2{,}3\,\mathrm{mm}$ de elongación del cilindro.

Un controlador validado solo en simulación (*Model-in-the-Loop*, MiL) no considera los efectos del hardware: la discretización, la resolución finita del ADC y del PWM, y la latencia de comunicación. Este trabajo evalúa esos efectos mediante un esquema *Processor-in-the-Loop* (PiL). El modelo de un actuador se ejecuta en Simulink y el controlador se ejecuta en un Arduino UNO R3; ambos se comunican por UART. Se comparan tres casos: MiL, PiL con sensores y actuadores ideales, y PiL con no idealidades tomadas de las hojas de datos.

II. Modelo de planta en Simulink

El modelo dinámico del actuador parte del descrito en [1]. Está compuesto por una válvula direccional proporcional 4/3, un cilindro de doble efecto Bosch Rexroth CDH1 40/28 [2], tuberías de $16\,\mathrm{mm}$ de diámetro y $0{,}8\,\mathrm{m}$ de longitud, y una masa móvil con fricción viscosa. Sus parámetros se resumen en la Tabla I. La posición x se mide desde el vástago totalmente retraído ($x = 0$).

Respecto a [1], el MiL se reajustó. Con la sintonización original ($K_p = 6{,}7$, $K_i = 1$), la válvula solo saturaba para errores mayores a $0{,}15\,\mathrm{m}$, y la acción integral, con una constante de tiempo cercana a $7\,\mathrm{s}$, producía una cola de asentamiento de unos $12\,\mathrm{s}$. Por ello se elevó K_p a 60, manteniendo $K_i = 1$, y se limitó el comando a $u \in [-1, 1]$, el rango normalizado de apertura de la válvula. Además, se aumentaron la presión de suministro a $160\,\mathrm{bar}$ y el caudal nominal de la válvula a $40\,\mathrm{L/min}$. Con $K_p = 60$, la válvula permanece abierta casi todo el recorrido y empieza a cerrar a unos $1{,}7\,\mathrm{cm}$ ($1/K_p$) del objetivo. La frecuencia natural hidráulica estimada es $\omega_h \approx 390\,\mathrm{rad/s}$, con $\zeta \approx 0{,}06$.

La presión de suministro queda por debajo de la nominal del cilindro ($250\,\mathrm{bar}$), y la velocidad máxima alcanzada, de aproximadamente $0{,}40\,\mathrm{m/s}$, queda por debajo de su límite de $0{,}5\,\mathrm{m/s}$ [2]. Para un escalón de $0\,\mathrm{m}$ a $0{,}40\,\mathrm{m}$ aplicado en $t = 0{,}5\,\mathrm{s}$, que deja $5\,\mathrm{cm}$ de margen al tope de carrera, el MiL con el controlador discreto de la Sec. V presenta un tiempo de subida de $0{,}73\,\mathrm{s}$, un tiempo de asentamiento al $2\,\%$ de $1{,}05\,\mathrm{s}$ y un sobreimpulso de $3{,}43\,\%$ ($13{,}7\,\mathrm{mm}$), con un error en estado estacionario de $0{,}016\,\mathrm{mm}$.

Tabla I
Parámetros del modelo de planta

Parámetro	Valor
Cilindro [2]	CDH1 40/28
Área del pistón A_1	$12{,}57\,\mathrm{cm}^2$
Área anular A_3	$6{,}41\,\mathrm{cm}^2$
Carrera L_{cyl}	$0{,}45\,\mathrm{m}$
Posición inicial x_0	$0\,\mathrm{m}$ (retraído)
Presión de suministro p_s	$160\,\mathrm{bar}$
Módulo volumétrico efectivo β_e	$750\,\mathrm{MPa}$
Masa móvil m	$20\,\mathrm{kg}$
Fricción viscosa b	$1000\,\mathrm{N\,s/m}$
Tuberías (diámetro, longitud)	$16\,\mathrm{mm}$, $0{,}8\,\mathrm{m}$
Caudal nominal de la válvula Q_N	$40\,\mathrm{L/min}$ a $35\,\mathrm{bar}$
Constante de tiempo de la válvula τ_v	$2\,\mathrm{ms}$
Retardo de la válvula T_d	$1\,\mathrm{ms}$

III. Arquitectura PiL y comunicación

III-A. Hardware y software

La planta se simula en MATLAB/Simulink R2026a con un solver de paso fijo `ode4` de $0{,}1\,\mathrm{ms}$. El controlador se ejecuta

en un Arduino UNO R3 (ATmega328P, 16 MHz) conectado por USB a 115 200 baud (puerto COM6). En el modelo, un bloque MATLAB Function de periodo $T_s = 10\,\text{ms}$ invoca en cada muestreo una función de MATLAB que aplica el modelo del sensor, arma la trama, la envía al MCU y espera su respuesta. La simulación queda así sincronizada paso a paso con el MCU (*lockstep*): Simulink no avanza hasta recibir $u[k]$, por lo que la latencia de la comunicación no se traslada a la planta. El mismo bloque, con la comunicación desactivada, ejecuta en MATLAB la misma ley de control y constituye la referencia MiL.

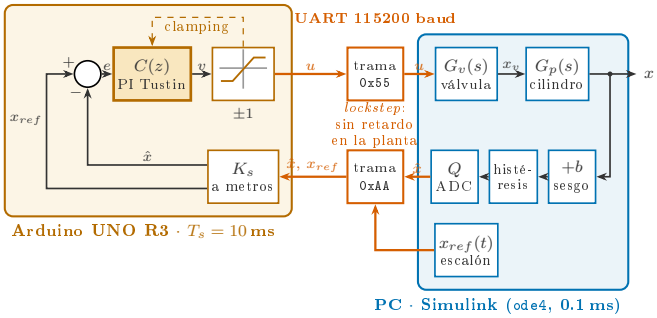


Figura 1. Arquitectura PiL: planta y modelos de sensor/actuador en Simulink, controlador en el MCU.

III-B. Protocolo y trama

La comunicación entre Simulink y el Arduino UNO R3 se realiza por UART con tramas binarias fijas de seis bytes en ambos sentidos (Fig. 2). La trama de Simulink al MCU contiene el encabezado `0xAA`, la posición medida y la referencia como enteros sin signo de 16 bits en unidades de 0,01 mm (byte bajo primero) y un *checksum*. La respuesta contiene el encabezado `0x55`, el comando u como entero con signo de 16 bits escalado por 32767, un contador de secuencia, un byte de estado y el *checksum*. El *checksum*, calculado como el XOR de los bytes anteriores, permite detectar errores de transmisión; si una trama llega corrupta, el MCU repite el último comando y lo indica en el byte de estado. Una trama con encabezado `0xA5` reinicia los estados del controlador antes de cada corrida.

Trama UART de 6 bytes (figura pendiente)

Figura 2. Trama UART de seis bytes entre Simulink y el Arduino.

III-C. Validación con el flujo PIL de Simulink

Como verificación independiente del protocolo propio, el lazo se repitió con el flujo *Processor-in-the-Loop* de Simulink (Embedded Coder y el *Simulink Support Package for Arduino Hardware*). El controlador se separó en el modelo `ctrl_pi.slx` (Fig. 4): un bloque Discrete PID Controller en forma PI, con integrador trapezoidal (Tustin), $K_p = 60$, $K_i = 1$, $T_s = 10\,\text{ms}$, saturación en ± 1 y anti-windup por *clamping*, es decir, la misma ley (2). Este modelo se referencia desde `PiL_oficial.slx` (Fig. 3) con un bloque Model en modo PIL: Embedded Coder genera el código C, lo compila para el ATmega328P y lo carga en la placa, y Simulink intercambia las entradas y la salida con el MCU por COM6 a 115 200 baud, también en *lockstep*. Como en el AVR un `double` ocupa 4 bytes, la interfaz del controlador es de precisión simple y el modelo host convierte los tipos a la entrada y a la salida. Las no idealidades del sensor de la Sec. VII aparecen en este modelo como bloques de Simulink en el camino de realimentación: sesgo, histéresis (*backlash*), cuantización y rango del ADC.

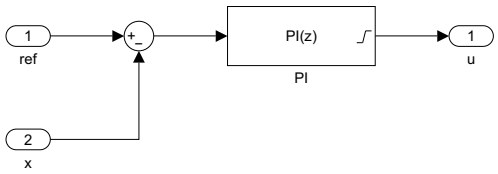


Figura 4. Modelo `ctrl_pi.slx`, compilado y ejecutado en el Arduino UNO R3.

IV. Interfaz sensor/actuador – MCU

IV-A. Sensor de posición

La posición se mide con el sensor magnetostrictivo integrado en el cilindro CDH1, de salida analógica 0 V a 10 V [2]. Un divisor 1:2 adapta esta señal al ADC de 10 bits del Arduino (5 V), cuya resolución en posición es

$$\Delta x = \frac{L_{cyl}}{2^N} = \frac{450\,\text{mm}}{1024} \approx 0{,}44\,\text{mm}, \tag{1}$$

muy por debajo del error de nivelación admisible de 2,3 mm.

IV-B. Electroválvula

Como actuador se usa la válvula proporcional 4/3 Bosch Rexroth 4WREE 6 en su versión de 32 L/min (a 10 bar por arista de control), compatible con el caudal nominal de 40 L/min a 35 bar del modelo. La válvula recibe un comando de $\pm 10\,\text{V}$ [5]. El Arduino genera u como un PWM de 8 bits centrado ($D = 50\,\% + 50\,\% \cdot u$), que un filtro RC y un amplificador llevan a $V_{cmd} = 10\,\text{V} \cdot u$. La resolución del comando es $\Delta u = 2/256 \approx 0{,}0078$ y, con la saturación en ± 1, se usa el rango completo de 256 cuentas. Con $K_p = 60$, un paso del ADC (0,44 mm) equivale a un cambio de 0.026 en u, unas tres cuentas del PWM, por lo que la cuantización de la medición se refleja directamente en el comando.

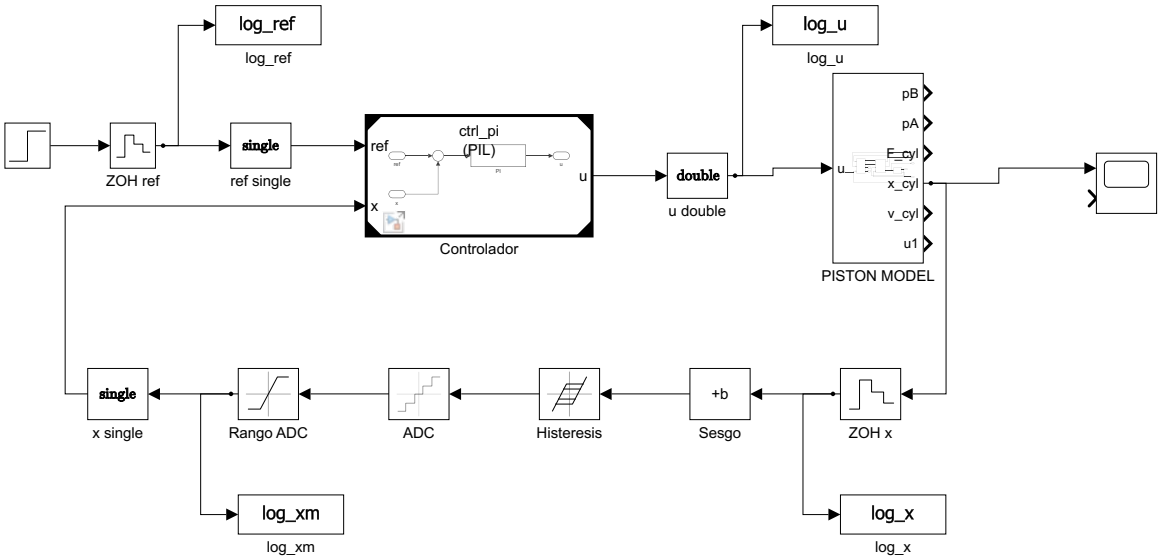


Figura 3. Modelo `PiL_oficial.slx`: el bloque Controlador referencia `ctrl_pi` en modo PIL; abajo, el modelo del sensor con sus no idealidades.

Tabla II
CADENA DE CONVERSIÓN DE SEÑALES EN EL ARDUINO UNO R3

Etapa	Conversión
Medición de posición (pistón → Arduino)	
Sensor del cilindro	0 mm a 450 mm → 0 V a 10 V
Divisor 1:2	0 V a 10 V → 0 V a 5 V
ADC 10 bits	0 V a 5 V → 0 a 1023 cuentas
Resolución	0,44 mm *por cuenta*
Comando de la válvula (Arduino → válvula)	
Controlador PI	$u \in [-1,\ 1]$
PWM 8 bits, 490 Hz	$u \rightarrow$ ciclo de trabajo de 0 a 100 %
Filtro RC	ciclo de trabajo → 0 V a 5 V
Amplificador	0 V a 5 V → ±10 V
Resolución	$\Delta u = 2/256 \approx 0{,}0078$

V. IMPLEMENTACIÓN DEL CONTROLADOR EN HARDWARE

El controlador se ejecuta en un Arduino UNO R3 [3], basado en el microcontrolador ATmega328P de 8 bits a 16 MHz [4]. El PI, sintonizado manualmente ($K_p = 60$, $K_i = 1$), se discretizó por el método de Tustin con un periodo de muestreo $T_s = 10\,\mathrm{ms}$ y se implementó en forma posicional:

$$\begin{aligned} I[k] &= I[k-1] + \tfrac{K_i T_s}{2}\big(e[k] + e[k-1]\big), \\ u[k] &= \mathrm{sat}_{\pm 1}\big(K_p\, e[k] + I[k]\big), \end{aligned} \tag{2}$$

con anti-windup por *clamping*: la integral no se actualiza cuando el comando está saturado y el error empuja en el mismo sentido de la saturación. Se descartó la forma incremental $u[k] = \mathrm{sat}(u[k-1] + q_0 e[k] + q_1 e[k-1])$. Con un escalón de 0,40 m, la acción proporcional ($K_p e \approx 24$) se recorta a 1 en la primera muestra y esa forma "olvida" el resto: el comando cae a cero cuando el pistón avanza $1/K_p \approx 1{,}7\,\mathrm{cm}$, y a partir de ahí solo la integral, lenta, mueve el pistón. En el MiL, con esa forma, el pistón llegó apenas a 38 mm en 3,5 s. El periodo de muestreo lo impone Simulink, que envía una trama con la posición cada T_s. El Arduino calcula $u[k]$ al recibir cada trama válida y responde de inmediato, por lo que el lazo queda sincronizado sin requerir un temporizador propio en el MCU.

El periodo $T_s = 10\,\mathrm{ms}$ es mucho menor que el tiempo de asentamiento del lazo (1,05 s). En cambio, la resonancia hidráulica estimada ($\omega_h \approx 390\,\mathrm{rad/s}$, unos 62 Hz) queda por encima de la frecuencia de Nyquist ($f_s/2 = 50\,\mathrm{Hz}$); si se excitara, aparecería en la medición como un alias cercano a 38 Hz. Como la válvula opera saturada durante casi todo el recorrido y el frenado es suave, no se espera excitarla; esto se verifica en los resultados.

VI. Implementación 1: sensores y actuadores ideales

Se aplicó un escalón de referencia de $0\,\mathrm{m}$ a $0{,}40\,\mathrm{m}$ en $t = 0{,}5\,\mathrm{s}$ y se simularon $4\,\mathrm{s}$. En el PiL ideal, la posición se envía al MCU sin modelo de sensor; la única diferencia con el MiL es que el controlador corre en el ATmega328P con aritmética de punto flotante de 32 bits y que la posición, la referencia y el comando viajan cuantizados por la trama ($0{,}01\,\mathrm{mm}$ y $1/32767$). La Fig. 5 compara ambas respuestas. A partir del escalón ($t \geq 0{,}5\,\mathrm{s}$), el error RMS entre ellas es de $0{,}0036\,\mathrm{mm}$ y el error máximo, de $0{,}0061\,\mathrm{mm}$; en las 401 tramas intercambiadas se detectaron 0 errores de *checksum*, igual que en las otras cuatro corridas PiL. Antes del escalón la diferencia es algo mayor: el transitorio inicial de presión desplaza levemente el vástago y lo lleva a posiciones negativas, que la trama, por usar enteros sin signo, recorta a cero. Este efecto queda fuera de la ventana de las métricas y no altera la respuesta al escalón.

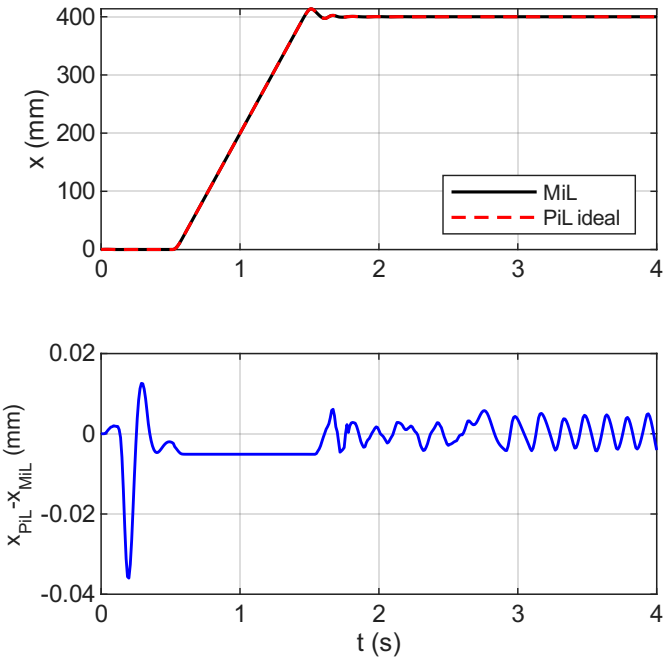


Figura 5. Respuesta al escalón de $0\,\mathrm{m}$ a $0{,}40\,\mathrm{m}$: MiL y PiL ideal.

VII. Implementación 2: sensores y actuadores no ideales

Se implementaron tres no idealidades en el modelo del sensor, del lado de Simulink, en el orden sesgo, histéresis (*backlash*) y cuantización, de modo que el MCU recibe la posición tal como la entregaría el ADC. Los valores del sensor se tomaron de la hoja de datos del fabricante [2] y se refieren a la carrera $L_{cyl} = 0{,}45\,\mathrm{m}$. El error de linealidad es de $\pm 0{,}02\,\%$ de la longitud de medición, es decir, $\pm 0{,}09\,\mathrm{mm}$, y se modela en el peor caso como un sesgo constante. La histéresis es de $0{,}004\,\mathrm{mm}$. La cuantización sigue (1) y es la no idealidad de medición de mayor magnitud. La repetibilidad del sensor ($\pm 0{,}0045\,\mathrm{mm}$) es dos órdenes de magnitud menor que la cuantización y no se modela. El límite de velocidad del fabricante ($0{,}5\,\mathrm{m/s}$) tampoco se incluye, ya que, como se mostró en la Sec. II, la velocidad máxima del MiL es de aproximadamente $0{,}40\,\mathrm{m/s}$. Se realizó una corrida con cada no idealidad por separado y una con las tres juntas.

Tabla III
No idealidades implementadas

No idealidad	Valor	Fuente	Modelo
Cuantización ADC	$L_{cyl}/2^N$	[4]	Redondeo a 1024 niveles
Error de linealidad	$+0{,}09\,\mathrm{mm}$	[2]	Sesgo constante
Histéresis del sensor	$0{,}004\,\mathrm{mm}$	[2]	Banda muerta (*backlash*)

VIII. Resultados y análisis comparativo

VIII-A. *Respuesta MiL vs PiL ideal vs PiL no ideal*

La Fig. 6 muestra la posición y el comando en los tres casos. Durante el recorrido, las tres curvas son indistinguibles: la válvula queda saturada en $u = 1$ desde el escalón hasta que el error baja de $1/K_p$, el pistón avanza a velocidad casi constante y el frenado produce el mismo sobreimpulso y las mismas oscilaciones amortiguadas. Esas oscilaciones son lentas, de pocos ciclos entre $1{,}5\,\mathrm{s}$ y $2\,\mathrm{s}$, y en u no aparece ninguna componente rápida que indique el alias de la resonancia hidráulica discutido en la Sec. V. Las diferencias solo se aprecian en el detalle cerca de la referencia: el PiL ideal se superpone al MiL, mientras que el PiL no ideal mantiene una pequeña oscilación sostenida alrededor de $400\,\mathrm{mm}$ y queda ligeramente por debajo de la referencia.

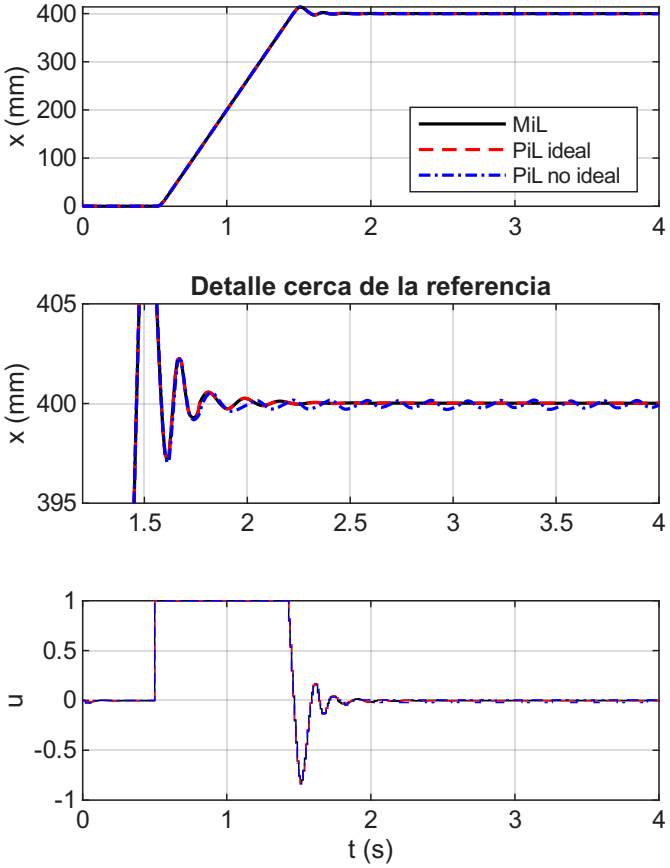


Figura 6. Posición y comando de control en los tres casos.

VIII-B. Métricas de desempeño

La Tabla IV resume las métricas de los tres casos, calculadas desde el escalón ($t \geq 0{,}5\,\mathrm{s}$); el error en estado estacionario es el promedio del error en el último segundo. El tiempo de subida ($0{,}73\,\mathrm{s}$) y el de asentamiento ($1{,}05\,\mathrm{s}$) coinciden en los tres casos con la resolución de T_s, y el RMS del comando es 0.528 en todos ellos, porque el recorrido está dominado por la saturación. El PiL ideal reproduce el MiL hasta la tercera cifra: $M_p = 3{,}42\,\%$ frente a $3{,}43\,\%$ y $e_{ss} = 0{,}016\,\mathrm{mm}$ en ambos. En el PiL no ideal el sobreimpulso sube a $3{,}47\,\%$ ($13{,}87\,\mathrm{mm}$) y el error en estado estacionario a $0{,}058\,\mathrm{mm}$, con un error RMS de $0{,}177\,\mathrm{mm}$ y un error máximo de $0{,}511\,\mathrm{mm}$ respecto al MiL.

Tabla IV
MÉTRICAS DE DESEMPEÑO

Métrica	Req.	MiL	PiL ideal	PiL no ideal
M_p (%)	< 10	3.43	3.42	3.47
t_r (s)	–	0.73	0.73	0.73
t_s 2 % (s)	–	1.05	1.05	1.05
e_{ss} (mm)	$< 2{,}3$	0.016	0.016	0.058
RMS(u)	–	0.528	0.528	0.528
RMSE vs MiL (mm)	–	–	0.0036	0.177
$e_{\mathrm{máx}}$ vs MiL (mm)	–	–	0.0061	0.511

VIII-C. Análisis de la degradación

El paso de MiL a PiL ideal introduce una diferencia de apenas $0{,}0036\,\mathrm{mm}$ RMS ($0{,}0061\,\mathrm{mm}$ como máximo). Como la simulación corre en *lockstep*, esta diferencia no proviene de la latencia de la UART, sino de la aritmética de punto flotante de 32 bits del AVR, frente a la de 64 bits de MATLAB, y de la cuantización de la trama ($0{,}01\,\mathrm{mm}$ en posición y referencia, $1/32767$ en el comando). Ninguna de las 2005 tramas de las cinco corridas PiL (401 por corrida) presentó errores de *checksum*, por lo que el canal de comunicación no aporta degradación.

La Tabla V y la Fig. 7 separan el aporte de cada no idealidad. La cuantización del ADC es la causa principal de la degradación: por sí sola produce un error RMS de $0{,}113\,\mathrm{mm}$ y un error máximo de $0{,}253\,\mathrm{mm}$ respecto al MiL, y genera la oscilación sostenida alrededor de la referencia, de amplitud menor a un paso del ADC ($0{,}44\,\mathrm{mm}$). El error de linealidad le sigue con $0{,}091\,\mathrm{mm}$ RMS, pero domina el error en estado estacionario ($0{,}074\,\mathrm{mm}$): como el PI regula la posición medida, la posición real tiende a quedar desplazada en el sesgo de $0{,}09\,\mathrm{mm}$, y la integral, con $K_i = 1$, aún no lo alcanza a los $4\,\mathrm{s}$. La histéresis de $0{,}004\,\mathrm{mm}$ es despreciable ($0{,}0062\,\mathrm{mm}$ RMS, $e_{ss} = 0{,}016\,\mathrm{mm}$, igual al ideal). Los efectos no se suman linealmente: con las tres juntas, el error RMS es de $0{,}177\,\mathrm{mm}$ y el error máximo de $0{,}511\,\mathrm{mm}$, pero el e_{ss} ($0{,}058\,\mathrm{mm}$) es menor que con el sesgo solo: como el sesgo ($0{,}09\,\mathrm{mm}$) es una fracción del paso del ADC ($0{,}44\,\mathrm{mm}$), el redondeo enmascara parte de él.

Aun en el caso no ideal, el controlador cumple ambos requisitos con amplio margen: $M_p = 3{,}47\,\%$ frente al límite de $10\,\%$ (margen de $6{,}53$ puntos porcentuales) y $e_{ss} = 0{,}058\,\mathrm{mm}$ frente a $2{,}3\,\mathrm{mm}$ (margen de $2{,}24\,\mathrm{mm}$, unas 40 veces por debajo del límite).

Tabla V
APORTE DE CADA NO IDEALIDAD (PIL, UNA POR CORRIDA)

Caso	M_p (%)	e_{ss} (mm)	RMSE (mm)	$e_{\mathrm{máx}}$ (mm)
Ideal	3.42	0.016	0.0036	0.0061
Cuantización	3.41	0.031	0.113	0.253
Linealidad	3.40	0.074	0.091	0.095
Histéresis	3.42	0.016	0.0062	0.012
Todas	3.47	0.058	0.177	0.511

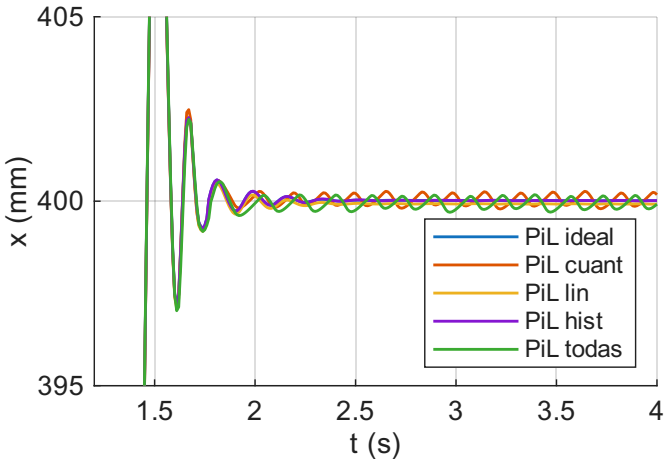


Figura 7. Detalle de la posición cerca de la referencia para cada no idealidad por separado y para las tres juntas.

VIII-D. Comparación con el PIL de Simulink

La Tabla VI resume los mismos seis casos corridos con el flujo PIL de Simulink (Sec. III-C). En este caso el MiL es el mismo modelo con el bloque Model en modo Normal. El PiL ideal coincide exactamente con él (error nulo en las 401 muestras): el código C generado reproduce bit a bit la simulación del controlador en precisión simple. Las conclusiones sobre las no idealidades se mantienen: la cuantización del ADC sigue siendo la dominante ($0{,}121\,\mathrm{mm}$ RMS), el error de linealidad es el que más eleva el e_{ss} ($0{,}076\,\mathrm{mm}$) y la histéresis es despreciable. Con las tres juntas, $M_p = 3{,}47\,\%$ y $e_{ss} = 0{,}047\,\mathrm{mm}$, de nuevo dentro de los requisitos con amplio margen.

Tabla VI
RESULTADOS CON EL FLUJO PIL DE SIMULINK

Caso	M_p (%)	e_{ss} (mm)	RMSE (mm)	$e_{\max}$ (mm)
MiL (Normal)	3.43	0.014	–	–
Ideal	3.43	0.014	0	0
Cuantización	3.41	0.024	0.121	0.256
Linealidad	3.40	0.076	0.089	0.092
Histéresis	3.42	0.014	0.0048	0.0098
Todas	3.47	0.047	0.160	0.521

IX. CONCLUSIONES

- El controlador PI ejecutado en el ATmega328P reproduce la simulación MiL: el error RMS entre ambas es de $0{,}0036\,\mathrm{mm}$ y el sobreimpulso y el error en estado estacionario coinciden ($3{,}42\,\%$ frente a $3{,}43\,\%$; $0{,}016\,\mathrm{mm}$). La aritmética de 32 bits y la cuantización de la trama no afectan el desempeño.
- El protocolo de seis bytes con *checksum* XOR resultó confiable: 2005 tramas intercambiadas en cinco corridas, sin ningún error.
- La cuantización del ADC de 10 bits es la no idealidad dominante ($0{,}113\,\mathrm{mm}$ RMS frente a $0{,}091\,\mathrm{mm}$ del error de linealidad y $0{,}0062\,\mathrm{mm}$ de la histéresis), mientras que el error de linealidad es el que más eleva el error en estado estacionario ($0{,}074\,\mathrm{mm}$).
- Con todas las no idealidades, el sistema cumple los requisitos con amplio margen: $M_p = 3{,}47\,\% < 10\,\%$ y $e_{ss} = 0{,}058\,\mathrm{mm} < 2{,}3\,\mathrm{mm}$.

Como limitaciones, el PiL se realizó para un solo pistón y no para las tres patas de MARLUP, y la ejecución en *lockstep* no representa la latencia real de la UART ni la variabilidad del tiempo de cómputo. Como trabajo futuro se propone un PiL en tiempo real, en el que el MCU imponga su propio periodo de muestreo, y extender la validación a las tres patas con el controlador de nivelación completo.

REFERENCIAS

[1] A. Salas, M. Solís y E. Valle, "MARLUP: Avance 2", 2026.
[2] Bosch Rexroth AG, "Hydraulic cylinder, mill type, series CDH1/CGH1," hoja de datos RE 17331/09.05, 2005.
[3] Arduino S.r.l., "Arduino UNO R3 Product Reference Manual," hoja de datos SKU A000066.
[4] Microchip Technology Inc., "ATmega328P 8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash," hoja de datos.
[5] Bosch Rexroth AG, "Proportional directional valves, types 4WRE and 4WREE, size 6 and 10," hoja de datos RE 29061/11.12, 2012.
[6] W. Wang, Y. Ning, Y. Zhang, P. Xu y B. Li, "Linear active disturbance rejection control with linear quadratic regulator for Stewart platform in active wave compensation system," *Applied Ocean Research*, vol. 156, art. 104469, 2025. [En línea]. Disponible: https://doi.org/10.1016/j.apor.2025.104469

APÉNDICE

ANEXO: REPRODUCIBILIDAD

Repositorio: https://github.com/aasalas/LAB-DE-CONTROL-GR5, commit `1f89753`. En `MODELOS/PISTON`: modelo `PiL.slx`, controlador del lado de Simulink `pil_ctrl.m`, código del MCU `pil_arduino/pil_arduino.ino`, carga del programa `subir_arduino.m`, corridas y métricas `run_pil_all.m` (todo el flujo con `correr_todo.m`) y datos crudos en `resultados/`. El PIL de Simulink (commit `8970620`) está en `PiL_oficial.slx` y `ctrl_pi.slx`, se arma con `pil_oficial_setup.m`, se corre con `run_pil_oficial.m` y sus datos están en `resultados/pil_oficial/`.